



Software Relevant Tools, Standards, and Code Versioning using GitHub

Name:	Lobos, Alyssa May F.	Date:	8/24/2026
Program:	Computer Engineering	Section:	B3

A. Learning Outcomes:

At the end of this laboratory, students should be able to:

- 1. Identify software development tools and standards used in software projects.
- 2. Create and manage a GitHub repository.
- 3. Apply Git version control commands.
- 4. Collaborate using GitHub features.
- 5. Demonstrate proper documentation and coding standards.

B. Laboratory Scenario:

You have been hired as a junior software developer in a software company. Your team uses GitHub for source code management and follows coding standards to maintain software quality. Your task is to create a project repository, manage versions of source code, and document your work according to industry standards.

C. Practical Tasks:

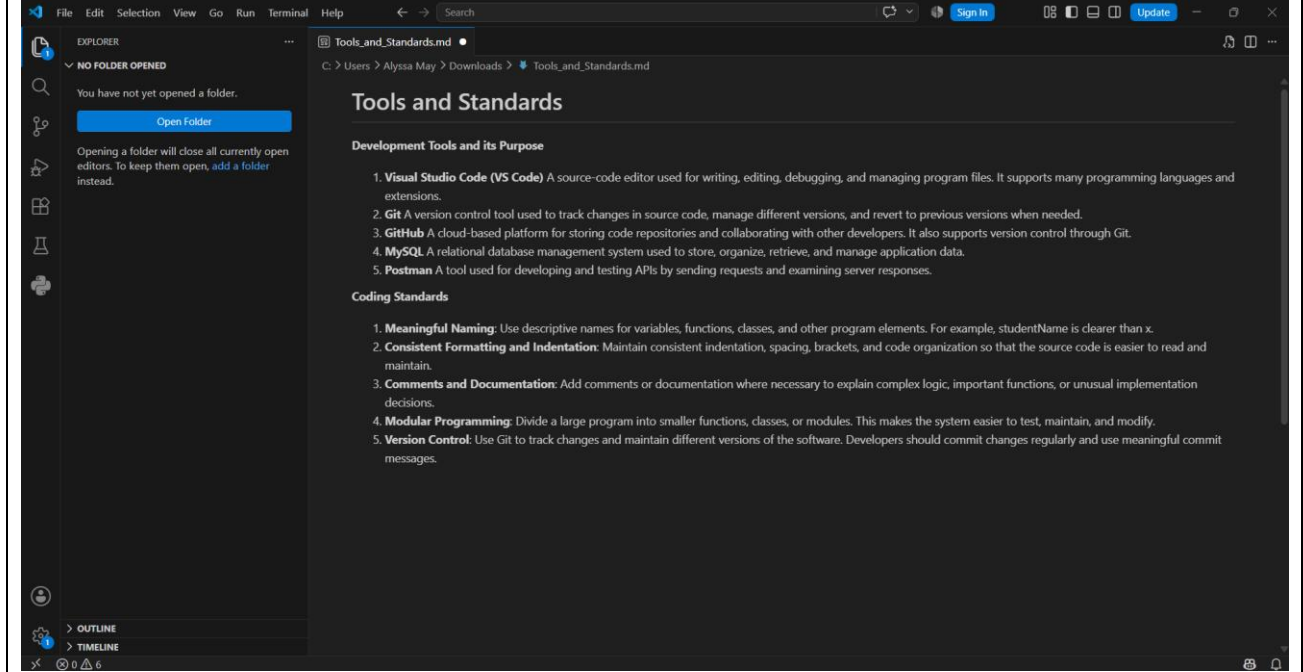
1. Software Development Tools and Standards

Create a document named Tools_and_Standards.md containing:

- a. List at least 5 development tools.
- b. Describe the purpose of each tool.
- c. Identify at least three coding standards or best practices used in software development.

(Please include the following: 1. Screenshots of the output)

Screenshot of Output



*Resize the image if necessary.

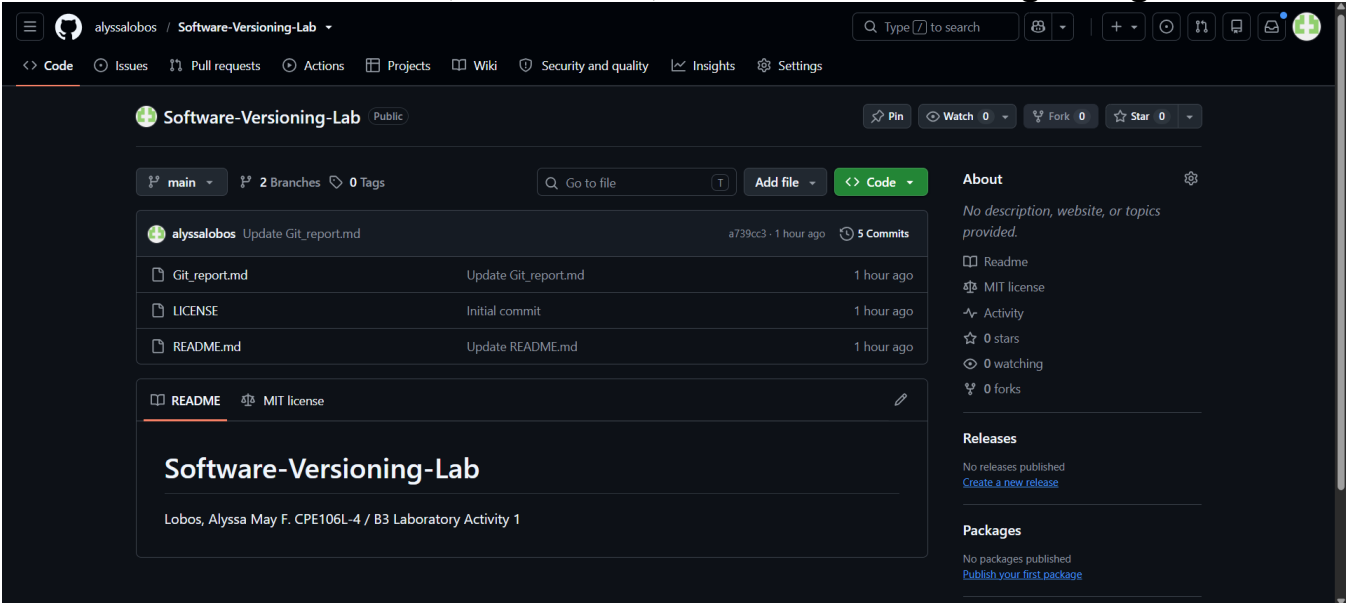
- 2. GitHub Repository Creation
 - a. Create a GitHub account (if none exists).
 - b. Create a public repository named: Software-Versioning-Lab
 - c. Add the following:
 - i. README.md
 - ii. LICENSE
 - iii. .gitignore
 - d. Upload this laboratory file in this repository once accomplished.
 - e. Submit GitHub repository link
 - f. Screenshot of repository homepage

(Please include the following: **1.** Screenshots of the output)

Screenshot of Output



Software Relevant Tools, Standards, and Code Versioning using GitHub



Link: <https://github.com/alyssalobos/Software-Versioning-Lab>

3. Git Version Control
- a. Perform the following Git commands and document them in a file named: (Git_Report.md)

i. Clone Repository

a) git clone <repository-url>

ii. Creating New Branch

a) git checkout -b feature-update

iii. Modify README.md

a) Add your name and laboratory information

iv. Commit Changes

a) git add .

b) git commit -m "Updated README with laboratory information"

v. Push Changes

a) git push origin feature-update

vi. Create Pull Request

a) feature-update → main

b. Screenshot the following:

i. Commands executed

ii. Pull Request

iii. Commit history

(Please include the following: **1.** Screenshots of the output)

Screenshot of Output

Software Relevant Tools, Standards, and Code Versioning using GitHub

```
MINGW64:/c/Users/Alyssa May/Software-Versioning-Lab

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~
$ git --version
git version 2.55.0.windows.5

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~
$ cd Desktop
bash: cd: Desktop: No such file or directory

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~
$ git clone https://github.com/alyssalobos/Software-Versioning-Lab.git
Cloning into 'Software-Versioning-Lab'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 14 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (14/14), done.

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~
$ cd Software-Versioning-Lab

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (main)
$ ls
Git_report.md  LICENSE  README.md

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (main)
$ git checkout -b feature-update
Switched to a new branch 'feature-update'

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ code .

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git status
On branch feature-update
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git add .

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git status
On branch feature-update
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git commit -m "Update README with laboratory information"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'Alyssa May@LAPTOP-01V3TTV2.(none)')

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$
```

```
Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git commit -m "Update README with laboratory information"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'Alyssa May@LAPTOP-01V3TTV2.(none)')

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git config --global user.name "Alyssa Lobos"

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git config --global user.email "amflobos@mymail.mapua.edu.ph"

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git config --global user.name
Alyssa Lobos

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git config --global user.email
amflobos@mymail.mapua.edu.ph

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git commit -m "Update README with laboratory information"
[feature-update 5675e0b] Update README with laboratory information
1 file changed, 5 insertions(+), 3 deletions(-)

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ |

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git push origin feature-update
info: please complete authentication in your browser...
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 413 bytes | 413.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature-update' on GitHub by visiting:
remote:   https://github.com/alyssalobos/Software-Versioning-Lab/pull/new/feature-update
remote:
To https://github.com/alyssalobos/Software-Versioning-Lab.git
 * [new branch]      feature-update -> feature-update

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ git push origin feature-update
Everything up-to-date

Alyssa May@LAPTOP-01V3TTV2 MINGW64 ~/Software-Versioning-Lab (feature-update)
$ |
```

*Resize the image if necessary.

Software Relevant Tools, Standards, and Code Versioning using GitHub

D. Findings, Observations, and Comments:

Guide Questions:

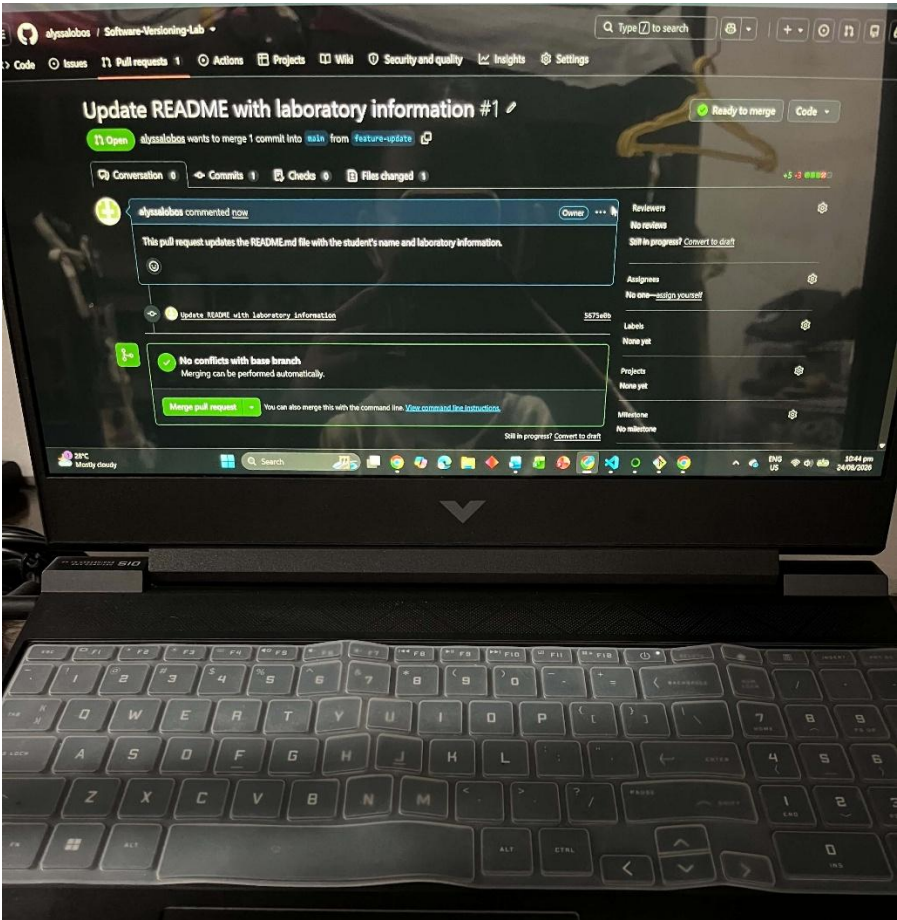
- 1. What additional steps are taken in this activity?
- 2. Are there any difficulties during the GitHub setup?
- 3. What are your findings, observations, and comments in this exercise?

1. Additional steps taken in this activity include creating a separate feature-update branch, modifying the README.md file, staging and committing the changes, pushing the branch to GitHub, and creating a Pull Request from feature-update to main. These steps demonstrate how changes can be developed separately and reviewed before being merged into the main branch.

2. Yes. One difficulty encountered was configuring the Git username and email before making the commit. Git required the user identity to be configured before allowing the changes to be committed. Another minor difficulty was understanding the Git commands and the difference between the local repository, branch, and remote GitHub repository. After configuring the Git identity and following the commands step by step, the process became easier to understand.

3. The exercise showed how Git and GitHub can be used to manage and track changes in a software project. I learned that branches allow developers to work on changes without directly affecting the main branch. I also observed that commits provide a record of changes, while pushing uploads the local branch to GitHub. Creating a Pull Request allows changes to be reviewed before being merged into the main project. Overall, the activity helped me understand the basic workflow of version control and the importance of organized collaboration in software development.

PICTURE/PROOF OF DOING THE ACTIVITY





Software Relevant Tools, Standards, and Code Versioning using Github



Software Relevant Tools, Standards, and Code Versioning using Github

E. Score Sheet

Criteria	Poor (25%)	Fair (50%)	Good (75%)	Excellent (100%)	Score
I. Completeness, Documentation, and Organization of Report	The report was incomplete, messy, and had many unanswered questions.	The report was complete, messy, and had many unanswered questions.	The report was complete, neat, and had 1 or 2 unanswered questions.	The report was complete, neat, and all the questions had been answered.	
II. Coding Design and Patterns	The program had inappropriate elements and structures, incorrect indentation, and poor program documentation.	The program had corrected indentation, but inappropriate elements and structures and poor program documentation.	The program had corrected indentation, adequate program documentation, but inappropriate elements and structures.	The program had appropriate elements and structures, correct indentation, and excellent program documentation.	
III. Findings, Observations, and Comments	The findings, observations, and comments were not based on the gathered data and results. All thoughts were inconsistent and unclear.	The findings, observations, and comments based on the gathered data and results but were not elaborated. Not all thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were mostly elaborated. Most of the thoughts were consistent and clear.	The findings, observations, and comments were based on the gathered data and results and were completely elaborated. All the thoughts were consistent and clear.	
IV. Grammar and Composition	The words used were inappropriate, wrong grammar usage, and poor sentence construction was observed.	The words used were appropriate; however, bad grammar and poor sentence construction were observed.	The words used were appropriate, had proper grammar usage, but poor sentence construction was observed.	The words used were appropriate, had proper grammar usage, and excellent sentence construction.	
SCORE:					

CHECKED			
Rating		Signature	
		Date	